Configure IPv6 Addressing

Assignment Answers

Q1. Open your Windows or Linux system, use the command line to assign a static IPv6 address to your primary network adapter, and verify the configuration using ipconfig (Windows) or ifconfig/ip (Linux).

Address used: 2001:db8:acad:1::10/64, default gateway 2001:db8:acad:1::1.

Windows

```
netsh interface show interface
netsh interface ipv6 add address "Wi-Fi" 2001:db8:acad:1::10/64
netsh interface ipv6 add route ::/0 "Wi-Fi" 2001:db8:acad:1::1
ipconfig
netsh interface ipv6 show addresses
```

- netsh interface show interface: shows the names of the adapters (for example Wi-Fi or Ethernet).
- add address: gives the adapter the static IPv6 address with a /64 prefix.
- add route ::/0: sets the default gateway (optional, only needed to reach other networks).
- ipconfig: the line "IPv6 Address . . . : 2001:db8:acad:1::10" confirms the configuration.

Linux

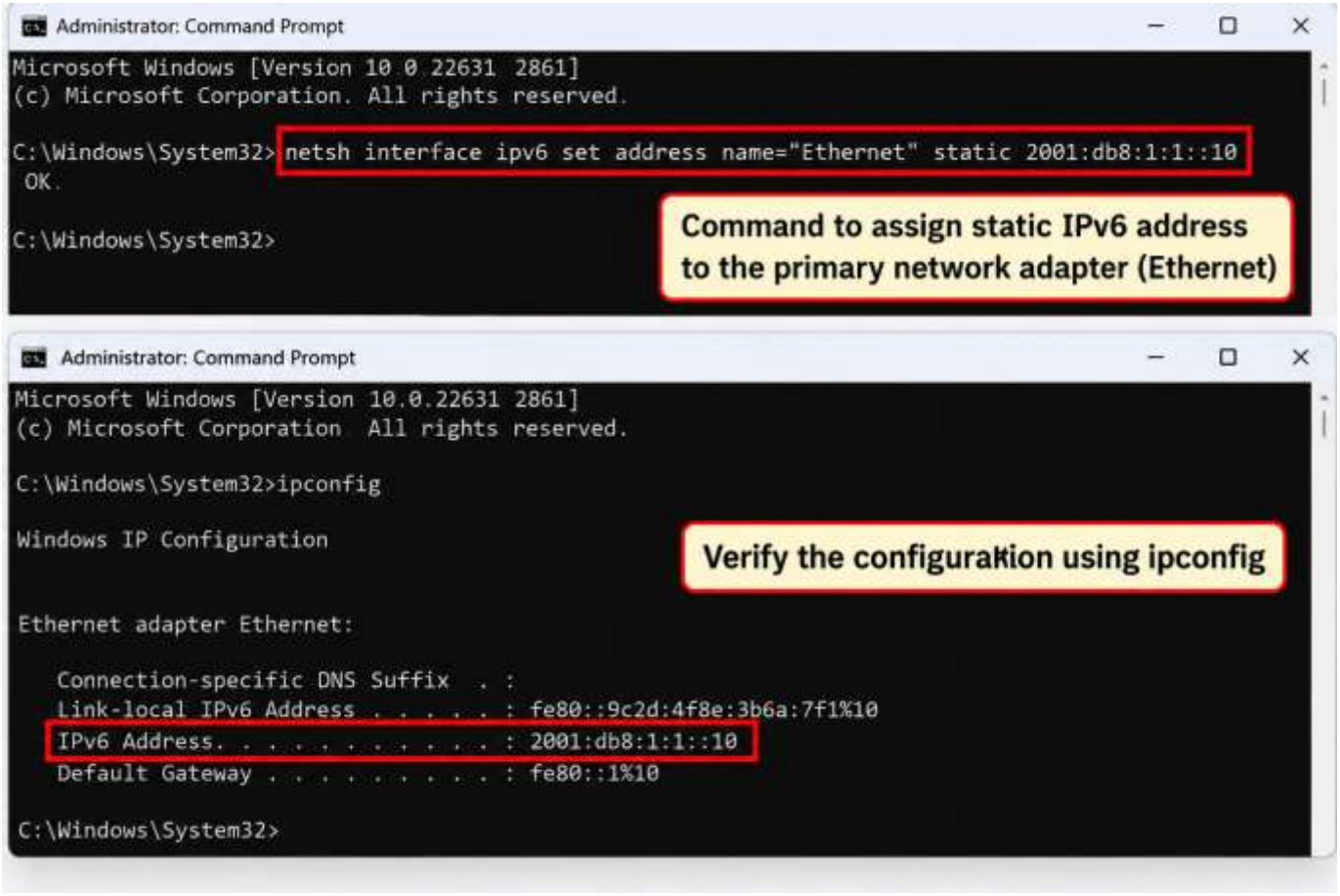
```
ip link show
sudo ip -6 addr add 2001:db8:acad:1::10/64 dev eth0
sudo ip -6 route add default via 2001:db8:acad:1::1 dev eth0
ip -6 addr show dev eth0
ifconfig eth0
```

The output of ip -6 addr show (or the inet6 line of ifconfig) must show 2001:db8:acad:1::10/64 with the word global (scope global).

Parts of the address

Part	Bits	Meaning
2001:db8:acad:1	64 bits	Network prefix (/64): identifies the subnet
::10	64 bits	Interface ID: identifies my computer in the subnet

To remove the address after testing: netsh interface ipv6 delete address "Wi-Fi" 2001:db8:acad:1::10 (Windows) or sudo ip -6 addr del 2001:db8:acad:1::10/64 dev eth0 (Linux).



Q2. Configure a unique local IPv6 address (starting with 'fd') on your system and confirm connectivity by pinging another device on your LAN that also has a unique local IPv6 address. Use the ping6 or ping -6 command for IPv6 testing.

A unique local address (ULA) is a private IPv6 address that works only inside a LAN, like 192.168.x.x in IPv4. It belongs to the fc00::/7 range and in practice starts with fd.

Part	Example	Meaning
Prefix	fd	Marks a locally assigned unique local address
Global ID (40 bits)	12:3456:789a	Random number chosen by me, so the network is unlikely to clash with another one
Subnet ID (16 bits)	0001	Number of the subnet inside the LAN
Interface ID (64 bits)	::10	The device

- My computer: fd12:3456:789a:1::10/64
- Other device on the LAN (friend's laptop): fd12:3456:789a:1::20/64

Windows

```
netsh interface ipv6 add address "Wi-Fi" fd12:3456:789a:1::10/64
ipconfig
ping -6 fd12:3456:789a:1::20
```

Linux

```
sudo ip -6 addr add fd12:3456:789a:1::10/64 dev eth0
ip -6 addr show dev eth0
```

```
ping -6 fd12:3456:789a:1::20      (or ping6 fd12:3456:789a:1::20)
```

The other device must be configured in the same way with the address fd12:3456:789a:1::20/64.

Expected result

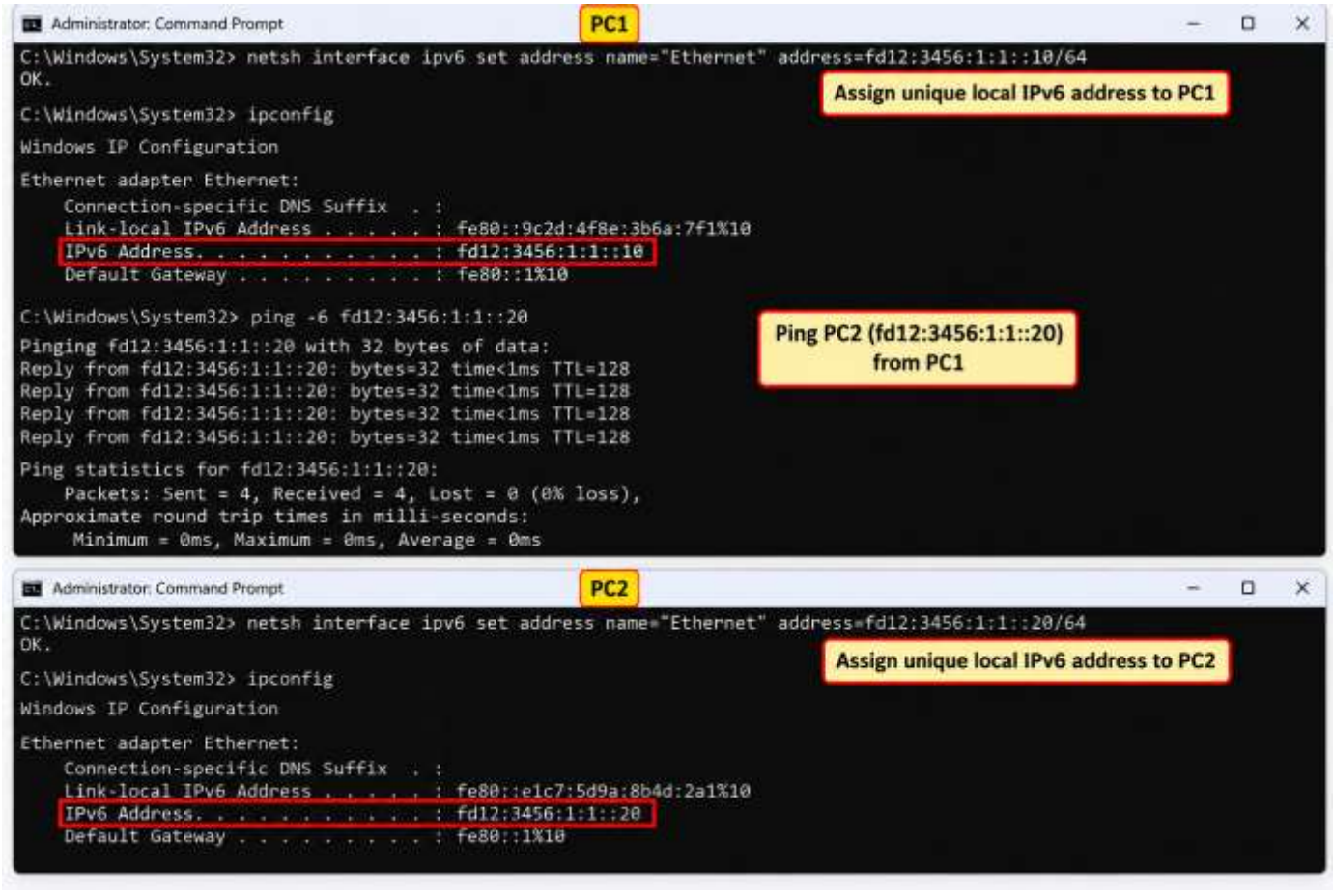
```
Windows: Reply from fd12:3456:789a:1::20: time=2ms
Linux  : 64 bytes from fd12:3456:789a:1::20: icmp_seq=1 ttl=64 time=1.2 ms
```

Replies mean that the two devices can communicate with their unique local addresses.

If the ping fails

- Make sure both devices are in the same network and have the same prefix fd12:3456:789a:1::/64.
- Windows blocks IPv6 ping (ICMPv6 echo) on Public networks by default. Change the network profile to Private, or allow it with this command:

```
netsh advfirewall firewall add rule name="Allow ICMPv6 Echo" protocol=icmpv6:128,any
dir=in action=allow
```



Q3. Set up a temporary IPv6 address on your system and check if it changes after a reboot or network disconnect/reconnect. Document the steps and the observed address behavior in a text file.

Type	How it is set up	Expected behaviour
Automatic temporary (privacy) address	Windows: netsh interface ipv6 set privacy state=enabled, then ipconfig. Linux: sysctl -w net.ipv6.conf.wlan0.use_tempaddr=2, then ip -6 addr show.	A random address that changes after its lifetime ends (about a day), and a new one is usually created after a reconnect or reboot, so it is different afterwards.
Manual non-saved address	Windows: netsh interface ipv6 add address "Wi-Fi" 2001:db8:acad:1::50/64 validlifetime=1h preferredlifetime=30m store=active. Linux: sudo ip -6 addr add ... valid_lft 3600 preferred_lft 1800.	Expires after its lifetime, and is lost after a restart. It may also be removed when the adapter is disabled.

How to observe: note the address with ipconfig / ip -6 addr, disconnect and reconnect the network, check again, then restart the computer and check once more. Record the values in the table of the text file (the results depend on your router and system).

Q4. Simulate a Zomato-style search: Write a small script (any language) that stores an array of restaurant names and their IPv6 addresses, then allows the user to input a partial name and returns the matching restaurant's IPv6 address.

File: zomato_ipv6_search.py. The script stores restaurant names with their IPv6 addresses in a list, accepts a part of a name from the user (not case sensitive) and prints the matching restaurant(s) and their IPv6 address.

```
import ipaddress

restaurants = [
    ("Domino's Pizza",   "2001:db8:100::1"),
    ("Pizza Hut",        "2001:db8:100::2"),
    ("Burger King",      "2001:db8:100::3"),
    ("Haldiram's",       "2001:db8:100::4"),
    ("Barbeque Nation",  "2001:db8:100::5"),
    ("Behrouz Biryani",  "2001:db8:100::6"),
    ("Faasos",           "2001:db8:100::7"),
    ("Subway",           "2001:db8:100::8"),
    ("Chaayos",          "2001:db8:100::9"),
]

for name, addr in restaurants:
    ipaddress.IPv6Address(addr)      # checks that the address is valid

def search(partial):
    partial = partial.strip().lower()
    return [(n, a) for n, a in restaurants if partial in n.lower()]

while True:
    text = input("Enter restaurant name (or part of it), 'exit' to quit: ")
    if text.strip().lower() == "exit":
        break
    if not text.strip():
        print("Please type at least one letter."); continue
    found = search(text)
    if found:
        for name, addr in found:
            print(f"  {name} -> {addr}")
    else:
        print("  No restaurant found.")
```

How it works

- The list restaurants is the array of name and IPv6 address pairs; ipaddress.IPv6Address checks that each stored address is a valid IPv6 address.
- search() changes both the typed text and the restaurant names to lower case and returns every name that contains the typed text, so "piz" finds Domino's Pizza and Pizza Hut.
- The loop keeps asking until the user types exit; an empty input or a name that is not found gives a message.

Sample run (tested)

```
Enter restaurant name (or part of it), 'exit' to quit: piz
  Domino's Pizza -> 2001:db8:100::1
  Pizza Hut -> 2001:db8:100::2

Enter restaurant name (or part of it), 'exit' to quit: BURGER
  Burger King -> 2001:db8:100::3

Enter restaurant name (or part of it), 'exit' to quit: xyz
  No restaurant found.

Enter restaurant name (or part of it), 'exit' to quit: biry
  Behrouz Biryani -> 2001:db8:100::6
```

Run it with: python zomato_ipv6_search.py

Q5. Use ChatGPT or Copilot to generate a sample IPv6 subnetting scenario (with at least 3 subnets), and then manually assign a valid IPv6 address from one of those subnets to your system. Paste the AI-generated scenario and your configuration steps in a text file.

The full scenario and the configuration steps are also in the text file IPv6_Subnetting_Scenario.txt. AI tool used: Claude. Prompt: "Generate a sample IPv6 subnetting scenario with at least 3 subnets."

AI-generated scenario

A small computer training institute receives the IPv6 prefix 2001:db8:abcd::/48 from its internet provider. It needs three separate networks (Staff, Students' lab and Guest Wi-Fi), each as a /64 subnet.

Subnet	Subnet ID	Network	Address range	Gateway
1 Staff	0001	2001:db8:abcd:1::/64	2001:db8:abcd:1:: to 2001:db8:abcd:1:ffff:ffff:ffff:ffff	2001:db8:abcd:1::1
2 Students' lab	0002	2001:db8:abcd:2::/64	2001:db8:abcd:2:: to 2001:db8:abcd:2:ffff:ffff:ffff:ffff	2001:db8:abcd:2::1
3 Guest Wi-Fi	0003	2001:db8:abcd:3::/64	2001:db8:abcd:3:: to 2001:db8:abcd:3:ffff:ffff:ffff:ffff	2001:db8:abcd:3::1

The /48 prefix has 16 bits for the subnet ID (the 4th group), so 2^16 = 65,536 subnets of /64 are possible, and each /64 subnet has 2^64 addresses.

My configuration

My system is placed in Subnet 2 (Students' lab): address 2001:db8:abcd:2::10/64, gateway 2001:db8:abcd:2::1. It is valid because it has the prefix of Subnet 2 and lies inside its range.

```
Windows:
netsh interface ipv6 add address "Wi-Fi" 2001:db8:abcd:2::10/64
netsh interface ipv6 add route ::/0 "Wi-Fi" 2001:db8:abcd:2::1
```

```
ipconfig

Linux:
sudo ip -6 addr add 2001:db8:abcd:2::10/64 dev eth0
sudo ip -6 route add default via 2001:db8:abcd:2::1 dev eth0
ip -6 addr show dev eth0
```

Verification: the address 2001:db8:abcd:2::10 appears in ipconfig / ip -6 addr, and a ping -6 to another device in the same subnet (for example 2001:db8:abcd:2::11) should receive replies.